

SZENARIO_1_Record_ETL

Szenario 1 : Record-Flow für ETL und Qualitätsprüfung

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login admin N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0 Fertiges Referenz-Lab auf der Canvas: **“Szenario 1 - Record ETL & Qualitaet”**

Didaktischer Mehrwert: Aus einem Demo-Flow wird ein echter **ETL-/Qualitätssicherungs-Flow**. Statt FlowFiles nur durchzureichen, arbeiten wir **record-orientiert**: NiFi liest die Datei als strukturierte Datensätze (über einen Record Reader mit Schema), prüft jeden Datensatz gegen ein Schema, filtert fachlich per SQL, schreibt den sauberen Pfad in die DB und stellt ungültige Datensätze in Quarantäne.

Flow:

```
GetFile —> ValidateRecord —valid—> QueryRecord —"gefiltert"—> PutDatabaseRecord
(sauberer Pfad)
                        |
                        |invalid—————|—————> PutFile (Quarantäne)
```

Das Record-Konzept (Kernidee)

Klassische Prozessoren (ReplaceText etc.) sehen die FlowFile als **Byte-Strom**. Record-Prozessoren sehen sie als **Liste von Datensätzen** mit einem **Schema**. Dazu braucht es Controller Services: - **Record Reader** (hier `CSVReader`) : parst CSV in Records, kennt das Schema. - **Record Writer** (hier `CSVRecordSetWriter`) : schreibt Records wieder raus.

Vorteil: Validierung, SQL-Abfragen und DB-Insert pro Datensatz : ohne die Datei zeilenweise zu zerlegen.

Vorbereitung (einmalig, ist bereits eingerichtet)

Zieltabelle in SQLite:

```
CREATE TABLE auftraege (auftrag_id INTEGER, kunde TEXT, betrag REAL, prioritael TEXT);
```

Avro-Schema (definiert Pflichtfelder + Typen):

```
{ "type": "record", "name": "auftrag", "fields": [
  { "name": "auftrag_id", "type": "int" },
  { "name": "kunde", "type": "string" },
  { "name": "betrag", "type": "double" },
  { "name": "prioritaet", "type": "string" } ] }
```

Beispieldaten data/samples/auftraege_mix.csv (4 gültige + 1 ungültiger):

```
auftrag_id,kunde,betrag,prioritaet
1001,Mueller GmbH,149.90,hoch      <- gültig, betrag>=50 -> DB
1002,Schmidt AG,29.50,normal       <- gültig, betrag<50  -> herausgefiltert
1003,Weber KG,210.00,hoch          <- gültig, betrag>=50 -> DB
2001,Fischer SE,abc,hoch           <- betrag kein double -> Quarantäne
2002,Klein OHG,75.00,normal        <- gültig, betrag>=50 -> DB
```

Controller Services (im Prozessgruppen-Kontext)

⚙ der Process Group → Tab **Controller Services** → +

Reader-Auftrag (CSVReader): - Schema Access Strategy = Use 'Schema Text' Property -
Schema Text = (das Avro-Schema oben) - Treat First Line as Header = true

Writer-Auftrag (CSVRecordSetWriter): - Schema Access Strategy = Inherit Record Schema

Beide **enable** (⚡).

Der **DBCConnectionPool** (SQLite) wird vom Root geerbt : muss nicht neu
angelegt werden.

Die Prozessoren im Detail

1. GetFile : liest eingehende CSV/JSON

- Input Directory = \$N/data/input/szenario1
- File Filter = *.*.csv
- Keep Source File = false, Polling Interval = 2 sec

2. ValidateRecord : prüft Schema und Pflichtfelder

- Record Reader = **Reader-Auftrag**
- Record Writer = **Writer-Auftrag**
- Schema Access Strategy = Use Reader's Schema
- Strict Type Checking = true (erzwingt korrekte Typen : betrag=abc wird ungültig)
- Allow Extra Fields = false
- Relationships: **valid**, **invalid**, failure

Datensätze, die nicht zum Schema passen (falscher Typ, fehlendes Pflichtfeld),
gehen über **invalid**; der Rest über **valid**.

3. QueryRecord : filtert/projiziert fachlich per SQL

- Record Reader = **Reader-Auftrag**, Record Writer = **Writer-Auftrag**
- Dynamische Property (mit + anlegen): Name gefiltert, Wert:

```
SELECT * FROM FLOWFILE WHERE betrag >= 50
```

- Der **Property-Name** wird zur **Relationship** (gefiltert). So kann man mehrere Abfragen = mehrere Ausgänge definieren.
- Relationships: **gefiltert** (unsere Query), **original**, failure

Hier könnte man auch projizieren, z.B. `SELECT auftrag_id, kunde FROM FLOWFILE` oder aggregieren. SQL über den Datenstrom : das ist die Stärke von QueryRecord.

4. PutDatabaseRecord : schreibt den sauberen Pfad in die DB

- Record Reader = **Reader-Auftrag**
- Database Connection Pooling Service = **DBCPConnectionPool**
- Statement Type = `INSERT`, Table Name = `auftraege`, Translate Field Names = `true`

5. PutFile : Quarantäne für ungültige Datensätze

- Directory = `$N/data/output/szenario1_quarantaene`
- Create Missing Directories = `true`, Conflict Resolution = `replace`

Beziehungen bewusst verdrahten

Von	Relationship	Nach	Bedeutung
GetFile	success	ValidateRecord	Datei rein
ValidateRecord	valid	QueryRecord	schema-konforme Datensätze
ValidateRecord	invalid	PutFile (Quarantäne)	Schema-/Typfehler
ValidateRecord	failure	terminieren	technischer Fehler
QueryRecord	gefiltert	PutDatabaseRecord	betrag>=50, sauber
QueryRecord	original	terminieren	unveränderte Eingabe (hier nicht gebraucht)
QueryRecord	failure	terminieren	
PutDatabaseRecord	success/failure/retry	terminieren	Endknoten
PutFile	success/failure	terminieren	Endknoten

Stolperstein (im Aufbau aufgetreten): Hängt man QueryRecords `original` ebenfalls an die Quarantäne, landen **gültige** Daten fälschlich dort. `original` ist die **unveränderte Kopie** der Eingabe : für diesen Flow terminieren.

Starten, einwerfen, beobachten

```
# Tabelle leeren (sauberer Start)
sqlite3 $N/data/db/fmea.db "DELETE FROM auftraege;"
# Process Group starten (Rechtsklick -> Start), dann:
```

```
cp $N/data/samples/auftraege_mix.csv $N/data/input/szenario1/
sleep 8
```

Ergebnis prüfen:

```
sqlite3 $N/data/db/fmea.db "SELECT * FROM auftraege ORDER BY auftrag_id;"
ls -l $N/data/output/szenario1_quarantaene/
cat $N/data/output/szenario1_quarantaene/*
```

Verifiziertes Ergebnis (im Live-Test bestätigt)

Datenbank auftraege (3 Zeilen):

```
1001 | Mueller GmbH | 149.90 | hoch
1003 | Weber KG      | 210.00 | hoch
2002 | Klein OHG      | 75.00  | normal
```

Quarantäne (szenario1_quarantaene/auftraege_mix.csv):

```
auftrag_id,kunde,betrag,prioritaet
2001,Fischer SE,abc,hoch
```

Herausgefiltert (weder DB noch Quarantäne): 1002 Schmidt AG, 29.50 (betrag<50).

Damit sind alle drei Pfade demonstriert: 1. **gültig + fachlich relevant** → Datenbank 2. **gültig, aber gefiltert** → verworfen (QueryRecord-Bedingung) 3. **ungültig (Schema/Typ)** → Quarantäne

Erweiterungsideen für die Schulung

- **JSON statt CSV:** Reader auf `JsonTreeReader` umstellen : selber Flow, anderes Format.
- **Quarantäne anreichern:** vor `PutFile` ein `UpdateAttribute` mit `fehlergrund` + `ValidateRecord` "Validation Details Attribute Name" setzen, damit der Grund mitgeschrieben wird.
- **Mehrere Filter:** in `QueryRecord` weitere dynamische Properties = weitere Ausgänge (z.B. `hochpriorisiert = SELECT * FROM FLOWFILE WHERE prioritaet = 'hoch'`).
- **Projektion:** `SELECT auftrag_id, betrag FROM FLOWFILE` : nur relevante Spalten in die DB.

Reset

```
sqlite3 $N/data/db/fmea.db "DELETE FROM auftraege;"
find $N/data/input/szenario1 $N/data/output/szenario1_quarantaene -type f -delete
# Process Group stoppen
```

Häufige Stolpersteine

- **Alles geht in invalid** → Schema-Typen passen nicht zu den Daten (z.B. `int` vs. Dezimalzahl); Schema Text und CSV-Header prüfen.
- **PutDatabaseRecord invalid** → DBCP nicht enabled oder Tabelle/Spalten stimmen nicht.
- **QueryRecord ohne Ausgang** → die SQL-Property wird erst durch Anlegen zur Relationship; danach verbinden/terminieren.
- **Reader/Writer nicht enabled** → Controller Services erst aktivieren (⚡), sonst sind die Record-Prozessoren invalid.